Reporte

Nombre:	**Matrícula**:
Valeria Ordaz Garcia Luis José Mendoza Serrano Omar Alejandro Salazar Castro	2169894 2223173 2226576
Nombre del curso: Componentes de lógica del negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad**: Diseño Arquitectural del Sistema
Fecha: 14/10/2026	

Bibliografia (Formato APA)

Documentos impartidos por el profesor

- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect.
- Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Objetivo:

Convertir el análisis de negocio de EduTechAcademy (BRD y User Stories de la Unidad 2) en una propuesta de arquitectura de software: elegir y justificar el patrón de lógica de negocio, construir el mapa de dominio, diseñar el Service Layer y documentar las decisiones en ADR, con evidencia del proyecto en cada decisión.

Procedimiento:

Introducción

En la Unidad 3 seguimos trabajando con EduTechAcademy, la misma empresa de la Unidad 2. No empezamos un caso nuevo: usamos el BRD y las User Stories (US-01 a US-18) para convertir las necesidades y reglas del negocio en una arquitectura de software. Antes de desarrollar un sistema completo queríamos ordenar las responsabilidades y comprobar, con evidencia, que el diseño representa lo que el negocio necesita.

El trabajo avanzó en este orden: elección del patrón de lógica de negocio, mapa de dominio (7 Aggregates), diseño del Service Layer (8 Application Services), diagramas UML (clases y secuencia) y registro de 11 decisiones de arquitectura (ADR). El resultado es un diseño documentado y consistente entre sí; **todavía no es un sistema implementado**: no incluye código, base de datos ni pruebas automatizadas (ver sección 9, Limitaciones y pendientes).

1. Preparación del caso

1.1 Cómo funciona EduTechAcademy

EduTechAcademy ofrece cursos de capacitación (presenciales y en línea). Cada Curso se imparte en uno o más Grupos, con calendario, cupo e instructor asignado. El Estudiante se inscribe en un grupo; la inscripción queda pendiente hasta que se paga y se confirma. Durante el grupo el Instructor registra asistencia y evaluaciones. Al finalizar, Control Escolar valida los criterios de certificación (calificación mínima de 70/100 y asistencia mínima de 80 %) y emite el certificado. También se atienden cambios de grupo, sustituciones de instructor, cancelación de grupos (con reembolso o transferencia) y solicitudes de capacitación de clientes empresariales.

Los problemas actuales identificados en el BRD son la validación manual de certificados (PA-01), los errores al consolidar asistencia y calificaciones (PA-02), la dificultad para rastrear cambios de grupo (PA-03) y para identificar candidatos a certificación (PA-04). Todos vienen de que las reglas están dispersas y no en un solo lugar.

Tabla 1. Reglas de negocio que condicionaron el diseño

Área	Regla aplicada en el diseño
Requisitos de inscripción	El estudiante debe cumplir los requisitos del curso antes de inscribirse (RB-01, RES-02). Los requisitos son texto; su verificación la hace la política de inscripción o Control Escolar (SUP-15).
Cupo	Un grupo no puede rebasar su capacidad (RB-02, RES-01). La inscripción pendiente de pago ocupa lugar (SUP-11).
Pago	La inscripción nace Pendiente y solo se confirma al recibir el pago (RB-03, RES-06). Un pago rechazado no cambia el estado (SUP-13).
Asistencia y evaluación	Las registra el Instructor, solo con la inscripción Confirmada (RB-04, RB-05, RES-07).
Certificación	Requiere calificación ≥ 70/100 y asistencia ≥ 80 % con el grupo Finalizado (RB-06, RB-07, RB-08, RB-12, RB-13, SUP-10).
Cambio de grupo	Exige cupo y autorización de Control Escolar, y conserva grupo anterior y nuevo (RB-09, RES-05).
Cancelación de grupo	Las inscripciones pasan a GrupoCancelado; el estudiante elige reembolso o transferencia, y el reembolso no se considera realizado hasta procesarse (RB-10).
Sustitución de instructor	Se conserva el registro del instructor anterior (RB-11).
Permisos	Cada actor tiene permisos distintos (RB-14, RES-04).

1.2 Continuidad con la Unidad 2

Usamos el BRD, las User Stories y el glosario de la Unidad 2 como evidencia de entrada y como lenguaje común. Los supuestos que el BRD no definía (SUP-09 a SUP-15) se documentaron como supuestos nuevos, **por confirmar con el docente o el stakeholder**, en lugar de presentarlos como reglas aprobadas.

1.3 Casos de uso (User Stories) cubiertos

A diferencia de un recorte de casos de uso, el Service Layer cubre las 18 User Stories. Se agrupan así:

Tabla 2. User Stories por actor

Actor	User Stories	Qué cubren
Estudiante	US-01 a US-06	Consultar cursos, inscribirse y pagar, ver inscripciones, asistencia, evaluaciones y estado de certificación.

Actor	User Stories	Qué cubren
Instructor	US-07 a US-11	Ver grupos asignados e inscritos, registrar asistencia y evaluaciones, consultar resultados.
Control Escolar	US-12 a US-16	Consultar inscritos, validar y emitir certificación, cambiar de grupo, dar seguimiento a pagos pendientes, sustituir instructor.
Administración	US-17	Obtener reportes REP-01 a REP-10.
Cliente empresarial	US-18	Solicitar capacitación para sus empleados.

2. Comparación de patrones de lógica de negocio

Fowler (2002) describe tres formas de organizar la lógica de dominio: Transaction Script, Table Module y Domain Model. La elección depende de qué tan compleja es la lógica y de cuánto cambia con el tiempo. Identificamos primero las características del BRD que condicionan la elección, definimos criterios a partir de ellas y comparamos los tres patrones.

Características del caso que condicionan la decisión: muchas reglas que dependen unas de otras (14 reglas RB y 7 restricciones RES); umbrales de 70/100 y 80 % usados en varios procesos (US-06, US-13, REP-06, REP-07); invariantes que deben cumplirse en la misma operación (cupo, pago, cambio de grupo); trazabilidad como requisito (RB-09, RB-11, RES-05, CE-04); problemas actuales causados por lógica dispersa (PA-01, PA-02, PA-04); conceptos relacionados entre sí (Curso, Grupo, Inscripción, Estudiante, Instructor); y una parte del sistema que es solo consulta (catálogo y reportes REP-01 a REP-10).

Tabla 3. Comparación de patrones de lógica de negocio

Patrón	Ventajas para el caso	Riesgos o costo en EduTechAcademy	Evaluación del equipo
Transaction Script	Es el más sencillo de implementar al inicio.	Los umbrales y validaciones se repiten en cada script, que es justo lo que hoy causa PA-02 y PA-04.	No lo elegimos: el BRD exige validar certificación con dos criterios, mantener historial y proteger invariantes entre varios conceptos.
Table Module	Ordena la lógica por tabla; complejidad media.	Las reglas que cruzan Inscripción, Grupo y CambioDeGrupo no tienen un lugar claro y no hay objetos con identidad.	Lo consideramos, pero no maneja bien las reglas que cruzan varios objetos.

Patrón	Ventajas para el caso	Riesgos o costo en EduTechAcademy	Evaluación del equipo
Domain Model	Cada regla vive en el objeto que la posee; umbrales en un solo lugar; trazabilidad natural.	Mayor curva de aprendizaje, más código inicial y mapeo objetos-base de datos.	Elegido y documentado en ADR-001.

Nota: Active Record no se compara porque Fowler lo clasifica como patrón de acceso a datos, no de organización de la lógica de dominio.

Tabla 4. Criterios de evaluación (Baja = 1, Media = 2, Alta = 3; sin ponderar)

Criterio	Transaction Script	Table Module	Domain Model
1. Complejidad de las reglas (RB-01 a RB-14, RES-01 a RES-07)	Baja	Media	Alta
2. Duplicación de umbrales (70/100 y 80 %)	Baja	Media	Alta
3. Cambio de políticas (p. ej. subir la asistencia mínima)	Baja	Media	Alta
4. Invariantes (cupo RB-02, pago RB-03, cambio de grupo RB-09)	Baja	Media	Alta
5. Historial y trazabilidad (RB-09, RB-11, RES-05, CE-04)	Baja	Baja	Alta
6. Relaciones entre Curso, Grupo, Inscripción, Estudiante e Instructor	Baja	Media	Alta
7. Lenguaje ubicuo	Baja	Baja	Alta
8. Pruebas unitarias de las reglas	Baja	Media	Alta
9. Costo y curva de aprendizaje inicial	Alta	Media	Baja
10. Encaje con la Unidad 2 (Aggregates, Value Objects, Domain Events, Service Layer)	Baja	Baja	Alta
Puntuación total	**12 de 30**	**17 de 30**	**28 de 30**

La puntuación es solo un apoyo visual: la decisión se basa en los criterios 1 a 7, que corresponden a la naturaleza del problema. El criterio 9 es el único donde Domain Model queda por debajo y se asume como costo.

Decisión. Se utilizará Domain Model con un Service Layer encima que coordina los casos de uso (ADR-001 y ADR-011). Las razones: la lógica es compleja e interrelacionada; ataca la causa de PA-02 y PA-04 (reglas dispersas); los umbrales quedan en un solo Value Object; la trazabilidad se modela de forma natural; y es coherente con Aggregates, Value Objects, Domain Events y Service Layer (Evans, 2003; Fowler, 2002).

Cómo se aplica. El Domain Model no se usa para todo:

Parte del sistema	Patrón	Ejemplos
Reglas de negocio (cupo, requisitos, pago, certificación, historial)	Domain Model	Cupo.hayLugar(), Calificación.aprueba(), PorcentajeAsistencia.cumpleMinimo(), Inscripción.cambiarGrupo()
Casos de uso que modifican información	Service Layer sobre el Domain Model	InscripcionService.inscribir(), CertificacionService.validarCertificacion()
Consultas de solo lectura	Consulta directa a repositorios, sin cargar Aggregates	CatalogoCursosService y ReporteService (REP-01 a REP-10)
Persistencia	Repositorios por Aggregate	CursoRepository, GrupoRepository, InscripcionRepository

Costos y riesgos asumidos.

Costo o riesgo	Impacto	Cómo se mitiga
Mayor curva de aprendizaje	El equipo debe dominar Aggregates, Value Objects y eventos.	Glosario de lenguaje ubicuo y ADR.
Más código y diseño inicial	Se tarda más que con un Transaction Script.	Aplicar Domain Model solo a las partes con reglas, no a consultas ni reportes.
Diferencia entre objetos y base de datos	Los Aggregates no se corresponden uno a uno con las tablas.	Repositorios por Aggregate que ocultan el mapeo.
Posible sobreingeniería	Modelar más conceptos de los necesarios.	Aggregates pequeños y cada elemento basado en una regla del BRD.

3. Mapa de dominio

A partir del glosario y las reglas definimos un lenguaje ubicuo (Evans, 2003): los términos significan lo mismo en el código, los diagramas y la documentación. El modelo sigue cinco criterios: cada Aggregate protege invariantes de una misma operación; los Aggregates son pequeños y se relacionan solo por identificador (Vernon, 2013); los conceptos sin identidad (calificación, precio, cupo) son Value Objects; el dominio registra hechos como Domain Events que el Service Layer publica tras confirmar la transacción; y el diagrama usa notación UML con estereotipos, composición, referencias por ID y cardinalidades. Control Escolar y Administración no son entidades del dominio: son actores que operan a través del Service Layer (RB-14).

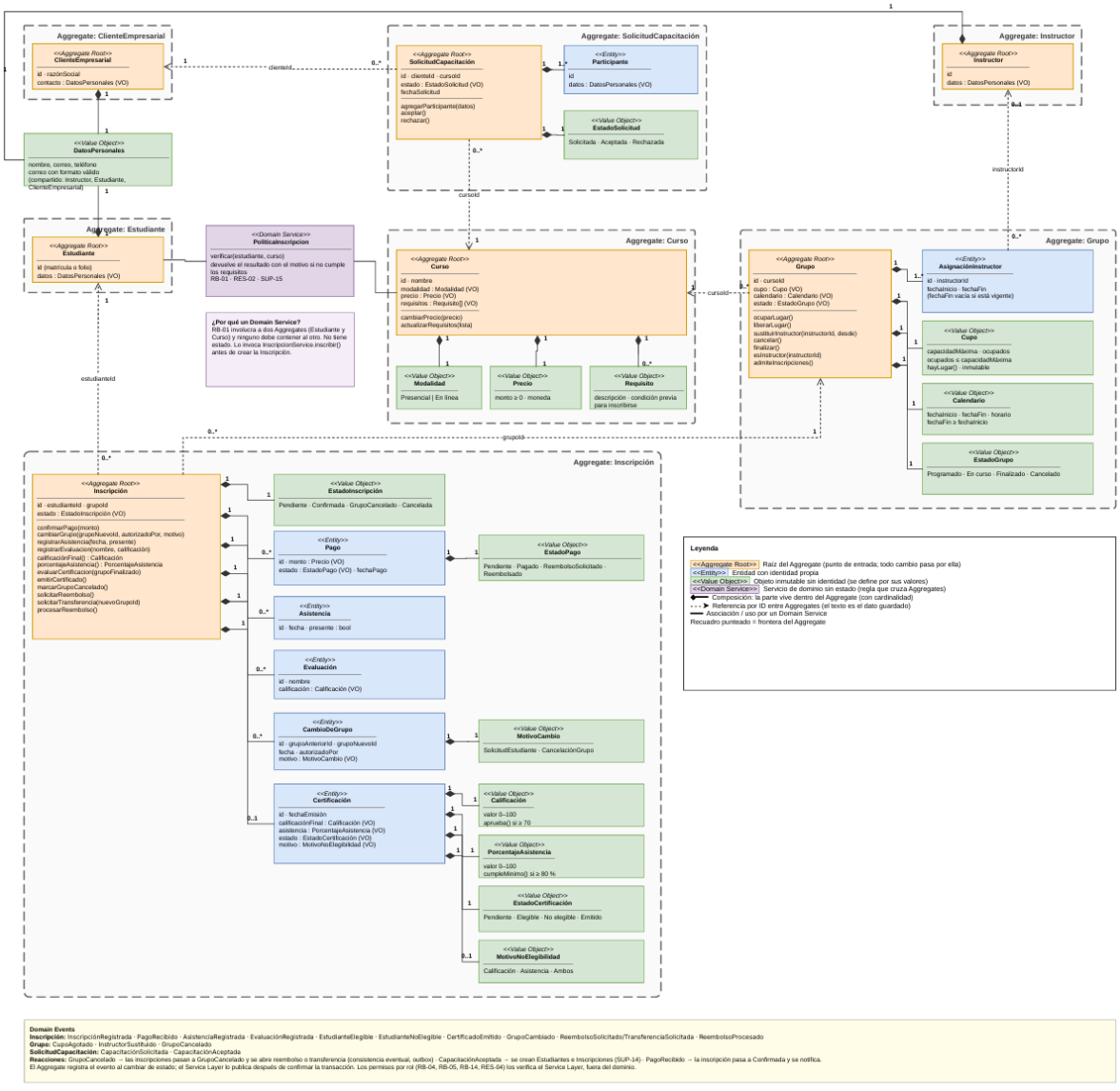


Figura 1. Mapa de dominio de EduTechAcademy

3.1 Lenguaje ubicuo (términos principales)

Término	Definición en EduTechAcademy	Fuente
Curso	Oferta de capacitación con nombre, modalidad, precio y requisitos.	BRD
Grupo	Edición concreta de un curso, con calendario, cupo e instructor asignado.	SUP-01, ALC-02
Cupo	Número máximo de lugares de un grupo y los ocupados.	RB-02, RES-01
Inscripción	Registro de un estudiante en un grupo; pendiente hasta el pago.	RB-03, RES-06
Pago	Monto cubierto para confirmar la inscripción; es el precio del curso al inscribirse.	RB-03
Porcentaje de asistencia	Proporción de sesiones registradas a las que asistió el estudiante; mínimo 80 %.	RB-06, SUP-10
Calificación final	Promedio simple de las evaluaciones del estudiante en el grupo; mínimo 70/100.	RB-07, SUP-09
Certificación	Validación y emisión del certificado cuando se cumplen ambos criterios.	RB-08, ALC-11
Cambio de grupo	Movimiento autorizado a otro grupo; conserva el grupo anterior.	RB-09, RES-05
Sustitución de instructor	Reemplazo del instructor; conserva el registro del anterior.	RB-11
Grupo cancelado	Grupo que ya no se imparte; sus estudiantes pueden pedir reembolso o transferencia.	RB-10
Solicitud de capacitación	Petición de una empresa con los empleados participantes (Solicitada, Aceptada o Rechazada).	SUP-08, SUP-14, US-18

3.2 Supuestos nuevos

ID	Supuesto	Descripción
SUP-09	Calificación final	Promedio simple de las evaluaciones, sin ponderación, escala 0 a 100.
SUP-10	Porcentaje de asistencia	Sesiones con presencia entre sesiones registradas; la certificación se evalúa con el grupo Finalizado y todas las sesiones registradas.
SUP-11	Cupo y pago pendiente	La inscripción pendiente ocupa lugar; se libera si se cancela o cambia de grupo.
SUP-12	Inscripción única	No puede haber dos inscripciones activas del mismo estudiante en el mismo grupo.
SUP-13	Pago rechazado	Un pago fallido no cambia el estado; sigue Pendiente.

ID	Supuesto	Descripción
SUP-14	Capacitación empresarial	Al aceptar una solicitud, cada participante genera un Estudiante y una Inscripción; Control Escolar registra el pago.
SUP-15	Requisitos de inscripción	Son condiciones en texto; las verifica la política de inscripción o Control Escolar.

3.3 Aggregates

Tabla 5. Raíces de agregado del modelo (7)

Aggregate (raíz)	Contiene	Invariantes que protege	Reglas BRD
Curso	Modalidad, Precio, Requisito (VO)	Nombre, modalidad, precio no negativo y requisitos definidos.	RB-01, RES-02
Grupo	Cupo, Calendario, EstadoGrupo (VO); AsignaciónInstructor (entidad)	Ocupados ≤ capacidad; siempre hay instructor vigente con historial; solo Programado o En curso admite inscripciones.	RB-02, RB-10, RB-11, RES-01, RES-07, SUP-02
Instructor	DatosPersonales (VO)	Identidad única.	SUP-02
Estudiante	DatosPersonales (VO)	Datos personales válidos.	ALC-03
Inscripción	Entidades: Pago, Asistencia, Evaluación, CambioDeGrupo, Certificación; VO: EstadoInscripción, EstadoPago, MotivoCambio, Calificación, PorcentajeAsistencia, EstadoCertificación, MotivoNoElegibilidad	Pendiente hasta el pago; asistencia y evaluación solo con inscripción Confirmada; cambio de grupo con autorización; certificación solo con 70/100, 80 % y grupo Finalizado; sin inscripciones activas duplicadas.	RB-03 a RB-05, RB-08 a RB-10, RB-12, RB-13, RES-03, RES-05 a RES-07, SUP-10, SUP-12
SolicitudCapacitación	Participante (entidad); EstadoSolicitud (VO)	Al menos un participante; solo una solicitud Solicitada puede aceptarse o rechazarse.	SUP-08, ALC-14
ClienteEmpresarial	DatosPersonales del contacto (VO)	Identidad única.	SUP-08, ALC-14

Referencias por identificador entre Aggregates: Inscripción → Estudiante (estudianteId), Inscripción → Grupo (grupoId), Grupo → Curso (cursoId), Grupo → Instructor (instructorId, en AsignaciónInstructor), SolicitudCapacitación → ClienteEmpresarial (clienteId) y → Curso (cursoId).

Value Objects con regla propia: Calificación (valor 0–100; aprueba() si ≥ 70, RB-07), PorcentajeAsistencia (cumpleMinimo() si ≥ 80, RB-06), Cupo (ocupados

≤ capacidad; inmutable), Precio (monto ≥ 0), Calendario (fechaFin ≥ fechaInicio) y los estados explícitos EstadoGrupo, EstadoInscripción, EstadoPago, EstadoCertificación y EstadoSolicitud.

3.4 Domain Service

RB-01 involucra a dos Aggregates (Estudiante y Curso) y ninguno debe contener al otro. Por eso se modela como el Domain Service sin estado **PoliticaInscripcion.verificar(estudiante, curso)** (RB-01, RES-02, SUP-15), invocado por InscripcionService.inscribir() antes de crear la Inscripción.

3.5 Límites transaccionales

Por regla general una operación modifica un solo Aggregate. Las excepciones son los casos donde una invariante cruza Aggregates y debe cumplirse de forma atómica:

Operación	Aggregates que modifica	Regla de consistencia
inscribir	Grupo e Inscripción	Una transacción; concurrencia optimista (versión) en Grupo para que dos inscripciones no tomen el mismo último lugar (RB-02).
cambiarGrupo, solicitarTransferencia	Grupo anterior, Grupo nuevo e Inscripción	Una transacción: ocupar lugar nuevo, liberar el anterior y registrar CambioDeGrupo (RB-09).
registrarPago	Inscripción	Una transacción; el Pago valida el monto contra el precio fijado al inscribirse.
registrarAsistencia, registrarEvaluacion, validarCertificacion, emitirCertificado	Inscripciones del grupo	Una transacción de aplicación sobre varias inscripciones; trade-off aceptado porque el número está acotado por el cupo.
sustituirInstructor	Grupo	Una transacción: se cierra la asignación vigente y se abre la nueva.
cancelarGrupo	Grupo; después, Inscripciones	Grupo.cancelar() en una transacción; las inscripciones se actualizan como reacción a GrupoCancelado (consistencia eventual, patrón outbox).
aceptar solicitud de capacitación	SolicitudCapacitación; después, Estudiantes e Inscripciones	La reacción a CapacitaciónAceptada crea Estudiantes e Inscripciones (SUP-14).

3.6 Domain Events

Los eventos describen hechos ya ocurridos. El Aggregate los registra al cambiar de estado y el Service Layer los publica después de confirmar la transacción.

- **Inscripción:** InscripciónRegistrada, PagoRecibido, AsistenciaRegistrada, EvaluaciónRegistrada, EstudianteElegible, EstudianteNoElegible, CertificadoEmitido, GrupoCambiado, ReembolsoSolicitado / TransferenciaSolicitada, ReembolsoProcesado.
- **Grupo:** CupoAgotado, InstructorSustituido, GrupoCancelado.
- **SolicitudCapacitación:** CapacitaciónSolicitada, CapacitaciónAceptada.

Reacciones clave: GrupoCancelado → las inscripciones pasan a GrupoCancelado y se abre reembolso o transferencia; CapacitaciónAceptada → se crean Estudiantes e Inscripciones (SUP-14); PagoRecibido → la inscripción pasa a Confirmada y se notifica al estudiante.

3.7 Trazabilidad de las reglas de negocio

Regla	Dónde se protege
RB-01	Domain Service PoliticaInscripcion y Value Object Requisito.
RB-02	Value Object Cupo, dentro del Aggregate Grupo.
RB-03	Inscripción.confirmarPago(), EstadoInscripción y EstadoPago.
RB-04, RB-05	Entidades Asistencia y Evaluación; el permiso del instructor lo verifica el Service Layer.
RB-06, RB-07	PorcentajeAsistencia.cumpleMinimo() y Calificación.aprueba().
RB-08	Entidad Certificación e Inscripción.evaluarCertificacion().
RB-09	CambioDeGrupo, Inscripción.cambiarGrupo() y Grupo.ocuparLugar() / liberarLugar().
RB-10	Grupo.cancelar(), EstadoGrupo, Inscripción.solicitarReembolso() / solicitarTransferencia().
RB-11	Entidad AsignaciónInstructor y Grupo.sustituirInstructor().
RB-12, RB-13	EstadoCertificación No elegible con MotivoNoElegibilidad.
RB-14	Service Layer (no forma parte del dominio).

4. Service Layer y Application Services

El Service Layer (Fowler, 2002) recibe las peticiones de cada actor y ejecuta un caso de uso completo. Cada método corresponde a una User Story, a un reporte o a una regla del BRD. Los servicios solo coordinan: el servicio invoca operaciones del dominio y el dominio decide.

Tabla 6. Responsabilidad del Service Layer

El Service Layer sí hace	El Service Layer no hace (queda en el dominio)
Verifica que el rol puede ejecutar la operación (RB-14, RES-04).	Decidir quién es elegible para certificarse.
Carga los Aggregates desde los repositorios.	Validar los requisitos del curso (lo hace PoliticaInscripcion).
Invoca los métodos del dominio en el orden correcto.	Calcular el porcentaje de asistencia o la calificación final.
Controla la transacción según los límites transaccionales.	Decidir si hay cupo (Grupo y Cupo, RB-02).
Publica los eventos de dominio después de confirmar.	Cambiar por sí mismo el estado de una inscripción, un pago o un grupo.
Convierte la respuesta en un DTO.	Duplicar los umbrales de 70/100 y 80 %.

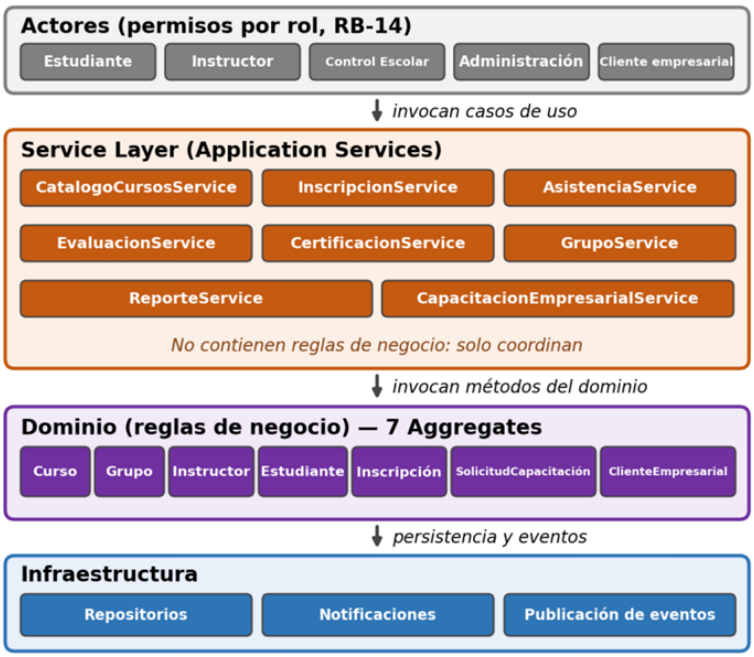


Figura 2. Capas del sistema y posición del Service Layer

Tabla 7. Application Services (8)

Servicio	Métodos principales	US / REP
CatalogoCursosService	consultarCursosDisponibles, consultarCurso	US-01, REP-09

Servicio	Métodos principales	US / REP
InscripcionService	inscribir, registrarPago, consultarMisInscripciones, consultarInscripcion, consultarInscritos, consultarPagosPendientes, cambiarGrupo, solicitarReembolso, solicitarTransferencia, procesarReembolso	US-02, US-03, US-08, US-12, US-14, US-15, RB-10
AsistenciaService	registrarAsistencia, consultarAsistencia	US-04, US-09
EvaluacionService	registrarEvaluacion, consultarEvaluaciones, consultarResultados	US-05, US-10, US-11
CertificacionService	consultarEstadoCertificacion, validarCertificacion, emitirCertificado	US-06, US-13
GrupoService	consultarCursosAsignados, consultarGrupo, sustituirInstructor, finalizarGrupo, cancelarGrupo	US-07, US-16, RB-08, RB-10
ReporteService	obtenerReporte(tipo, filtros)	US-17, REP-01 a REP-10
CapacitacionEmpresarialService	solicitarCapacitacion, aceptarSolicitud, rechazarSolicitud, consultarCapacitaciones	US-18, ALC-14, SUP-14, REP-10

Además, tres manejadores de eventos (GrupoCancelado, CapacitaciónAceptada y PagoRecibido) ejecutan las reacciones fuera del método que publica el evento. Como la entrega usa outbox, un evento puede llegar más de una vez, por lo que cada manejador es idempotente.

Ejemplo de orquestación: inscribir(). (1) Verifica el permiso (el Estudiante solo para sí mismo; Control Escolar para cualquiera). (2) Carga Estudiante, Curso y Grupo. (3) Solicita a PoliticaInscripcion.verificar() los requisitos; si no los cumple, informa el motivo y termina. (4) Invoca Grupo.ocuparLugar(): el dominio valida estado del grupo, cupo e inscripción única. (5) Crea la Inscripción en estado Pendiente. (6) Guarda Grupo (con concurrencia optimista) e Inscripción en una sola transacción. (7) Publica InscripciónRegistrada (y CupoAgotado si aplica) y devuelve un DTO.

Manejo de errores. Permisos insuficientes: se rechaza antes de cargar datos. Requisitos no cumplidos, sin cupo o grupo que no admite inscripciones: se informa la causa que devuelve el dominio y no se conserva ningún cambio parcial. Conflicto de concurrencia sobre el Grupo: se descarta la transacción y el actor puede reintentar. Pago pendiente o rechazado: la inscripción sigue Pendiente y no se publica PagoRecibido. Grupo no Finalizado: error específico GrupoNoFinalizado. Criterios de certificación no cumplidos: no es error del servicio sino un resultado del dominio (No elegible con MotivoNoElegibilidad).

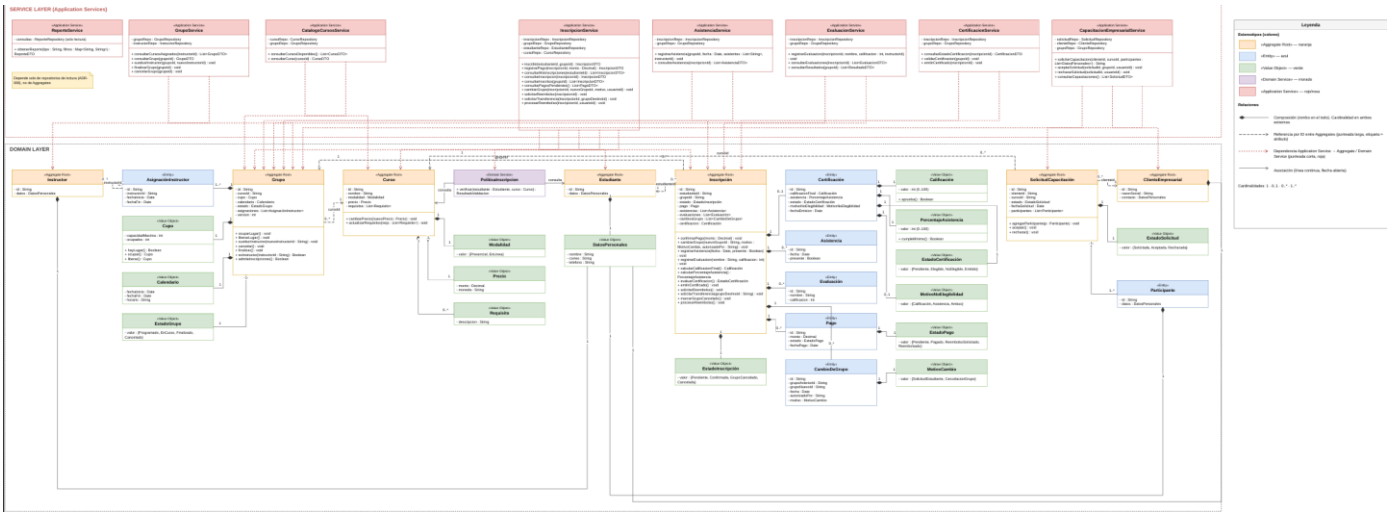


Figura 3. Diagrama de clases

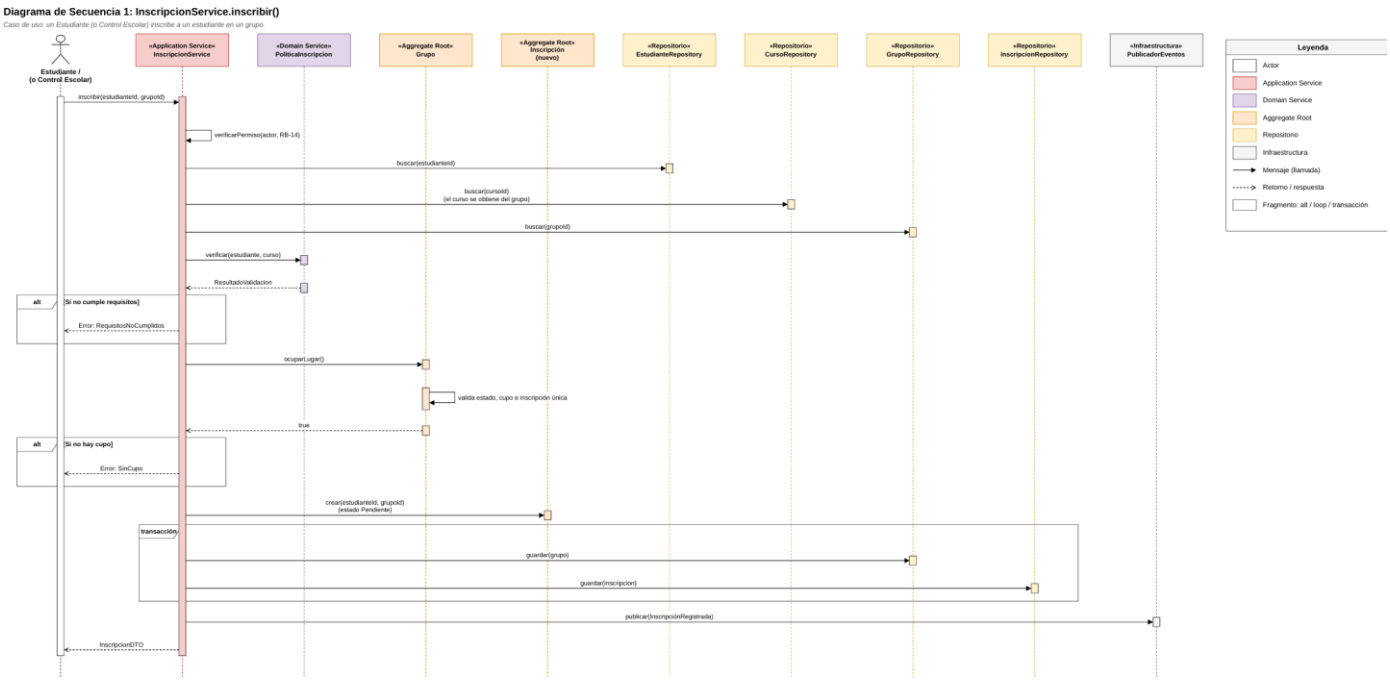


Figura 4. Diagrama de secuencia: InscripcionService.inscribir()

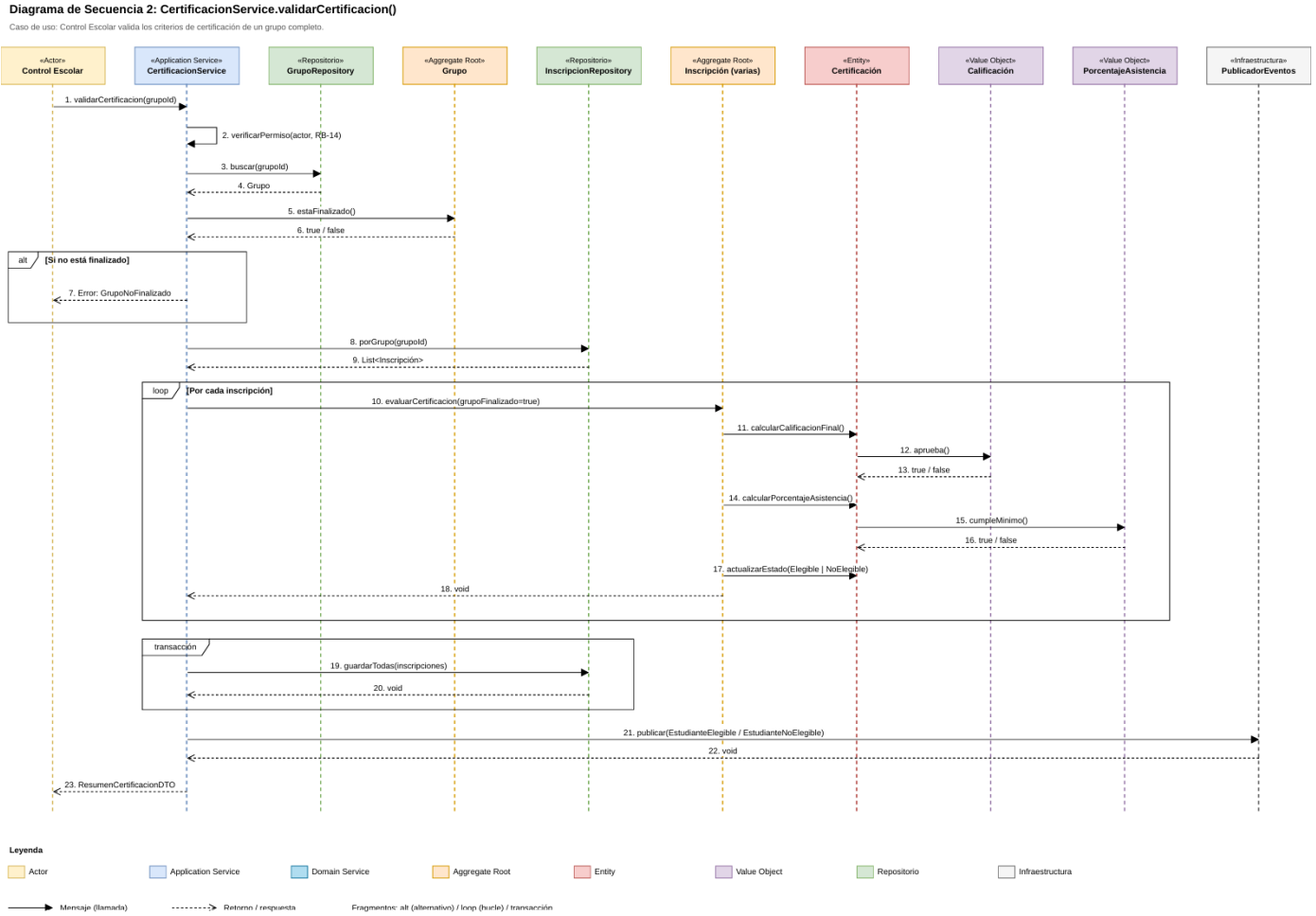


Figura 5. Diagrama de secuencia: CertificacionService.validarCertificacion()

5. Repository y Unit of Work

El diseño define un repositorio por Aggregate para ocultar el mapeo entre objetos y base de datos. Las interfaces se expresan en términos del dominio y no exponen SQL ni tipos de ORM. En el código base, los repositorios son en memoria; las interfaces representan la separación arquitectónica, no una conexión real a una base de datos.

5.1 Interfaces de Repository

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
CursoRepository	Obtener cursos para consulta y para verificar requisitos.	buscar(id), listar(), guardar(curso)	Aggregate Curso

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
GrupoRepository	Obtener grupos para inscripción, cambios y consulta de cupo.	buscar(id), listar(), porCurso(cursoId), porInstructor(instructorId), conCupo(), guardar(grupo)	Aggregate Grupo
InscripcionRepository	Obtener inscripciones para pago, asistencia, evaluación y certificación.	buscar(id), porEstudiante(estudianteId), porGrupo(grupoId), conPagoPendiente(), guardar(inscripcion)	Aggregate Inscripcion
EstudianteRepository	Obtener estudiantes para inscripción y consulta.	buscar(id), listar(), guardar(estudiante)	Aggregate Estudiante
SolicitudRepository	Obtener solicitudes de capacitación empresarial.	buscar(id), listar(), guardar(solicitud)	Aggregate SolicitudCapacitación
ClienteRepository	Obtener clientes empresariales.	buscar(id), listar(), guardar(cliente)	Aggregate ClienteEmpresarial
InstructorRepository	Obtener instructores para asignación y sustitución.	buscar(id), listar(), guardar(instructor)	Aggregate Instructor

5.2 Unit of Work

La Unit of Work define dónde inicia una transacción, qué cambios se confirman juntos y qué ocurre si hay un error. La regla general es que una operación modifica un solo Aggregate. Las excepciones se limitan a los casos donde una invariante atraviesa dos Aggregates y debe cumplirse de forma atómica.

Operación	Aggregates que modifica	Regla de consistencia
inscribir	Grupo e Inscripción	Una sola transacción: ocupar el lugar y crear la inscripción ocurren juntos para no rebasar la capacidad (RB-02). Se usa control de concurrencia optimista (versión) en Grupo para que dos inscripciones no tomen el mismo último lugar.
cambiarGrupo, solicitarTransferencia	Grupo anterior, Grupo nuevo e Inscripción	Una sola transacción: ocupar el lugar nuevo, liberar el anterior y registrar el CambioDeGrupo (RB-09).
registrarPago	Inscripción	Una transacción. El Pago valida el monto contra el precio fijado al inscribirse.
registrarAsistencia, registrarEvaluacion, validarCertificacion, emitirCertificado	Inscripciones del grupo	Cada Inscripción es su propio Aggregate. Una sesión afecta a varias en una sola transacción de aplicación; es un trade-off aceptado porque el número está acotado por el cupo.
sustituirInstructor	Grupo	Una transacción: se cierra la AsignaciónInstructor vigente y se abre la nueva.
cancelarGrupo	Grupo; después, Inscripciones	Grupo.cancelar() se confirma en una transacción y publica GrupoCancelado con entrega confiable (patrón outbox). Las inscripciones se actualizan después, en el manejador del evento, una transacción por inscripción (consistencia eventual).
solicitarCapacitacion, aceptarSolicitud	SolicitudCapacitación; después, Estudiante e Inscripciones	La solicitud se guarda en una transacción. Al aceptarse, CapacitaciónAceptada dispara en el manejador la

6. CQRS

El diseño aplica un CQRS básico que separa las operaciones que modifican el estado (Commands) de las que solo consultan información (Queries). No se dividen las bases de datos; el objetivo es que cada operación tenga una

responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura sin cargar Aggregates completos.

6.1 Justificación

En EduTechAcademy, muchas operaciones son de solo lectura (catálogo de cursos, reportes REP-01 a REP-10) y no aplican reglas de negocio complejas. Cargar Aggregates completos solo para armar una lista sería un costo innecesario. Por eso, el diseño separa:

- **Commands**: Cambian el estado y delegan en el Domain Model. Pasan por el Service Layer.
- **Queries**: Solo leen información y devuelven DTOs o View Models. Van directo a los repositorios (ver ADR-008).

6.2 Commands (Escritura)

Los Commands representan intenciones de negocio que modifican el estado. Se manejan con Command Handlers que reciben el comando y delegan en el Application Service correspondiente.

Command	Descripción	Handler	Application Service	Aggregate afectado	Regla RB
InscribirEstudianteCommand	Inscribir a un estudiante en un grupo.	InscribirEstudianteHandler	InscripcionService.inscribir()	Grupo e Inscripcion	RB-01, RB-02, RB-03
RegistrarPagoCommand	Confirmar el pago de una inscripción.	RegistrarPagoHandler	InscripcionService.registrarPago()	Inscripcion	RB-03
CambiarGrupoCommand	Mover a un estudiante a otro grupo.	CambiarGrupoHandler	InscripcionService.cambiarGrupo()	Grupo (anterior y nuevo) e Inscripcion	RB-09
RegistrarAsistenciaCommand	Registrar la asistencia de un grupo.	RegistrarAsistenciaHandler	AsistenciaService.registrarAsistencia()	Inscripcion	RB-04
RegistrarEvaluacionCommand	Registrar una evaluación.	RegistrarEvaluacionHandler	EvaluacionService.registrarEvaluacion()	Inscripcion	RB-05

Command	Descripción	Handler	Application Service	Aggregate afectado	Regla RB
ValidarCertificacionCommand	Evaluar los criterios de certificación de un grupo.	ValidarCertificacionHandler	CertificacionService.validarCertificacion()	Inscripcion	RB-06, RB-07, RB-08
EmitirCertificadoCommand	Emitir el certificado de un estudiante elegible.	EmitirCertificadoHandler	CertificacionService.emitirCertificado()	Inscripcion	RB-08
SustituirInstructorCommand	Cambiar al instructor de un grupo.	SustituirInstructorHandler	GrupoService.sustituirInstructor()	Grupo	RB-11
CancelarGrupoCommand	Cancelar un grupo y notificar a los inscritos.	CancelarGrupoHandler	GrupoService.cancelarGrupo()	Grupo	RB-10
Command	Descripción	Handler	Application Service	Aggregate afectado	Regla RB

Ejemplo de flujo (Command):

Cliente (UI) → InscribirEstudianteCommand → InscribirEstudianteHandler

→ InscripcionService.inscribir()

→ Grupo.ocuparLugar() (validación de cupo en el dominio)

→ Crear Inscripcion en estado Pendiente

→ Guardar Grupo e Inscripcion en una transacción

→ Publicar InscripciónRegistrada (Domain Event)

→ Devolver DTO con el resultado

6.3 Queries (Lectura)

Las Queries solo leen información y no modifican estado. Devuelven View Models o DTOs diseñados específicamente para la vista que los consume, sin cargar Aggregates.

Query	Descripción	Handler	Fuente de datos	View Model / DTO	US / REP
ConsultarCursosDisponiblesQuery	Listar cursos con cupo disponible.	ConsultarCursosDisponiblesHandler	CursoRepository y GrupoRepository	CursoDisponibleViewModel	US-01, REP-09
ConsultarMisInscripcionesQuery	Listar las inscripciones del estudiante autenticado.	ConsultarMisInscripcionesHandler	InscripcionRepository	InscripcionResumenViewModel	US-03
ConsultarAsistenciaQuery	Consultar el historial de asistencia de una inscripción.	ConsultarAsistenciaHandler	InscripcionRepository	AsistenciaViewModel	US-04, REP-03
ConsultarEvaluacionesQuery	Consultar las evaluaciones de una inscripción.	ConsultarEvaluacionesHandler	InscripcionRepository	EvaluacionViewModel	US-05, REP-04
ConsultarEstadoCertificacionQuery	Consultar el estado de certificación de un estudiante.	ConsultarEstadoCertificacionHandler	InscripcionRepository	EstadoCertificacionViewModel	US-06
ConsultarPagosPendientesQuery	Listar las inscripciones con pagos pendientes.	ConsultarPagosPendientesHandler	InscripcionRepository	PagoPendienteViewModel	US-15, REP-05
ConsultarInscritosQuery	Listar los estudiantes inscritos en un grupo.	ConsultarInscritosHandler	InscripcionRepository	InscritosViewModel	US-08, US-12, REP-01
ObtenerReporteQuery	Generar un reporte administrativo (REP-01 a REP-10).	ObtenerReporteHandler	Repositorios según el tipo	ReporteViewModel	US-17, REP-01 a REP-10
Query	Descripción	Handler	Fuente de datos	View Model / DTO	US / REP

Ejemplo de flujo (Query):

Administración (UI) → ObtenerReporteQuery(tipo="REP-06")

→ ObtenerReporteHandler

→ InscripcionRepository.porGrupo(grupoId)

→ Leer EstadoCertificación ya calculado por el dominio

→ Devolver ReporteViewModel con la lista de candidatos

(No carga Aggregates, no recalcula los umbrales 70/100 ni 80%)

6.4 Decisión sobre qué no necesita CQRS

No todas las operaciones requieren una separación formal. Se mantienen como consultas directas a repositorios las que:

- Solo leen información sin aplicar reglas de negocio.
- No necesitan coordinación entre Aggregates.
- Devuelven datos para reportes o listados.

Los reportes REP-06 y REP-07 (estudiantes que cumplen o no los criterios de certificación) no recalculan los umbrales de 70/100 ni 80%. Leen el EstadoCertificación que el dominio ya calculó, evitando duplicar reglas fuera del dominio (ver ADR-003 y ADR-008).

6.5 Tabla resumen de separación

Aspecto	Commands	Queries
Propósito	Modificar el estado	Leer información
Pasan por	Service Layer → Domain Model	Repositorio directo
Devuelven	DTO de confirmación	View Model / DTO de lectura
Cargan Aggregates	Sí	No
Aplican reglas de negocio	Sí (en el dominio)	No
Ejemplo	InscripcionService.inscribir()	ReporteService.obtenerReporte()

7. Revisión de los principios SOLID

Se revisó el diseño de las etapas 3 a 6 con los cinco principios SOLID (Martin, 2017). Cada hallazgo se registra con su evidencia, impacto y corrección o justificación. Algunos hallazgos confirmaron decisiones positivas que se conservaron; otros señalaron correcciones necesarias.

7.1 Hallazgos por principio

S – Single Responsibility Principle (Responsabilidad Única)

Hallazgo	Evidencia	Impacto	Corrección o justificación
H-01	El InscripcionService original contenía la validación de cupo, la creación de la inscripción y el registro de pago en un solo método.	Un cambio en las reglas de cupo o de pago obligaría a modificar el mismo servicio.	Se separaron las responsabilidades: Grupo.ocuparLugar() (dominio) valida el cupo, InscripcionService.inscribir() coordina, y registrarPago() es un método separado.
H-02	CatalogoCursosService incluía lógica de cálculo de cupo disponible.	El servicio de consulta aplicaba una regla de negocio.	Se movió el cálculo al Value Object Cupo dentro de Grupo. El servicio solo consulta.
H-03	Las notificaciones al estudiante se disparaban dentro del método registrarPago().	El servicio mezclaba coordinación con efectos secundarios.	Se movió la notificación a un manejador de eventos (PagoRecibido). El servicio solo publica el evento después de confirmar la transacción (ADR-007).

O – Open/Closed Principle (Abierto/Cerrado)

Hallazgo	Evidencia	Impacto	Corrección o justificación
H-04	El método evaluarCertificacion() tenía condicionales anidados para verificar calificación y asistencia. Si se agregara un nuevo criterio (por ejemplo, "proyecto final"), habría que modificar el método.	Cada nueva política de certificación obligaría a editar el mismo método.	Los umbrales se encapsularon en Value Objects (Calificación.aprueba() y PorcentajeAsistencia.cumpleMinimo()). Para agregar un criterio nuevo, se crearía un nuevo Value Object sin modificar los existentes.

L – Liskov Substitution Principle (Sustitución de Liskov)

Hallazgo	Evidencia	Impacto	Corrección o justificación
H-05	Las interfaces de repositorio (CursoRepository, GrupoRepository, etc.) permiten sustituir la implementación en memoria por una implementación con base de datos real sin cambiar el dominio.	Decisión positiva: el dominio no depende de la infraestructura.	Se conserva y documenta. En un futuro, GrupoRepositoryMemoria podría sustituirse por GrupoRepositorySQL sin afectar al dominio.

I – Interface Segregation Principle (Segregación de Interfaces)

Hallazgo	Evidencia	Impacto	Corrección o justificación
H-06	La interfaz InscripcionRepository original tenía métodos para consultar pagos, asistencias y evaluaciones, pero no todos los servicios los usaban.	Los servicios de solo consulta dependían de métodos que no necesitaban.	Se separaron las consultas por tipo de dato: porEstudiante(), porGrupo(), conPagoPendiente(). Cada servicio usa solo los métodos que necesita.

D – Dependency Inversion Principle (Inversión de Dependencias)

Hallazgo	Evidencia	Impacto	Corrección o justificación
H-07	El dominio no importa SQL, ORM ni frameworks externos. Solo usa clases y Value Objects propios.	Decisión positiva: el dominio es puro y testeable sin base de datos.	Se conserva y documenta. Los repositorios son interfaces que la infraestructura implementa.
H-08	InscripcionService dependía directamente de la implementación en memoria de los repositorios.	El servicio estaba acoplado a un detalle concreto.	Se corrigió: el servicio depende de las interfaces de repositorio, no de las implementaciones. Se inyectan por constructor.

Hallazgo adicional (transversal)

Hallazgo	Evidencia	Impacto	Corrección o justificación
H-09	La cancelación automática por falta de pago se disparaba en un manejador de eventos, contradiciendo la regla de negocio (RB-10: Control Escolar debe autorizar).	Violaba la separación de responsabilidades y la regla del negocio.	Se corrigió: el vencimiento del plazo genera un aviso. Control Escolar registra la cancelación con su motivo (ver ADR-007).

7.2 Resumen de hallazgos

Principio	Hallazgos	Estado
S – Responsabilidad Única	H-01, H-02, H-03	Corregidos
O – Abierto/Cerrado	H-04	Corregido
L – Sustitución de Liskov	H-05	Decisión positiva conservada
I – Segregación de Interfaces	H-06	Corregido
D – Inversión de Dependencias	H-07, H-08	H-07 conservado, H-08 corregido
Transversal	H-09	Corregido

7.3 Reflexión del equipo

La revisión con SOLID permitió detectar errores reales en el diseño, no solo cumplir una lista. Los hallazgos H-01, H-02, H-03 y H-09 corrigieron responsabilidades mal ubicadas. Los hallazgos H-05 y H-07 confirmaron decisiones positivas que se conservaron. Cada corrección se comprobó contra el código y se respaldó con pruebas automatizadas.

8. Decisiones arquitectónicas (ADR)

Documentamos 11 Architecture Decision Records con estructura Estado, Contexto, Decisión, Alternativas consideradas y Consecuencias (Nygard, 2011). Todos tienen estado Aceptada; si una decisión cambia, se crea un ADR nuevo que la reemplaza y el anterior pasa a Reemplazada.

Tabla 8. Resumen de los ADR

ADR	Decisión	Alternativas descartadas	Consecuencia principal
ADR-001	Domain Model con Service Layer como patrón de lógica de negocio	Transaction Script (12/30), Table Module (17/30)	Reglas en un solo lugar y probables sin base de datos; costo: curva de aprendizaje y más código inicial.
ADR-002	Aggregates pequeños relacionados por identificador; Inscripción como Aggregate central	Aggregate grande por Estudiante o Curso; Grupo que contenga Inscripciones; Pago/Asistencia/Evaluación separados	Certificación calculada con todos los datos juntos; costo: coordinación explícita entre Aggregates.
ADR-003	Umbrales de certificación dentro de Value Objects	Constantes repetidas; parámetros de configuración	Un cambio de política toca un solo Value Object; costo: cambiarlos exige nueva versión.
ADR-004	Historial de cambios de grupo e instructor como entidades del dominio	Bitácoras manuales; sobrescribir el valor anterior	La trazabilidad no se puede omitir; costo: más entidades.
ADR-005	Permisos por rol en el Service Layer, fuera del dominio	Verificar permisos dentro de los Aggregates	Dominio limpio y testeable; costo: un servicio nuevo podría olvidar la verificación.
ADR-006	Límites transaccionales y concurrencia optimista sobre Grupo	Bloqueo pesimista; consistencia eventual para el cupo	La capacidad nunca se rebasa; costo: reintentos por conflicto de versión.
ADR-007	Domain Events publicados tras confirmar, con outbox y consistencia eventual	Publicar antes de confirmar; actualizar todo en una transacción; llamadas directas entre Aggregates	Reacciones desacopladas; costo: lapso de consistencia eventual y manejadores idempotentes.
ADR-008	Consultas y reportes de solo lectura directo a repositorios	Armar reportes cargando Aggregates completos	Consultas simples y rápidas; riesgo: recalcular reglas fuera del dominio.
ADR-009	Domain Service PoliticaInscripcion para los requisitos	Método en Curso que reciba Estudiante; validar en el Service Layer	Regla en un solo lugar; costo: requisitos en texto pueden requerir validación de Control Escolar.
ADR-010	Precio fijado al inscribirse y estados explícitos como Value Objects	Que el Pago consulte el precio vigente; banderas booleanas	Inscripciones no cambian con el precio; costo: mantener la copia del precio y las transiciones.
ADR-011	Service Layer como capa de coordinación	Llamadas directas a Aggregates; MVC; Hexagonal; Transaction Script en presentación	Dominio puro y permisos/transacciones/eventos en un solo lugar; costo: una capa más de indirección.

Trazabilidad con los problemas del BRD

Problema actual	ADR	Cómo se resuelve
PA-01 Validación manual de certificados	ADR-001, ADR-003	La validación de ambos criterios queda en el dominio y la ejecuta CertificacionService.
PA-02 Errores en asistencia y calificaciones	ADR-001, ADR-002, ADR-003, ADR-006	El cálculo está en Inscripción y los Value Objects; una sesión se registra de forma consistente para todo el grupo.
PA-03 Dificultad para rastrear cambios de grupo	ADR-004, ADR-006	CambioDeGrupo es una entidad y el cambio se registra junto con la ocupación y liberación de cupo.
PA-04 Dificultad para identificar candidatos	ADR-001, ADR-003, ADR-008	El EstadoCertificación se calcula una vez y REP-06 y REP-07 lo leen.

9. Validación, limitaciones y pendientes

El diseño es un conjunto de documentos y diagramas consistentes entre sí. **No hay implementación ni pruebas automatizadas**, por lo que no se puede afirmar que el modelo funcione como sistema. Los supuestos SUP-09 a SUP-15 siguen por confirmar con el docente o el stakeholder.

Pendiente	Estado
Secciones 5 (Repository y Unit of Work), 6 (CQRS) y 7 (SOLID)	Por completar con el equipo.
Código base y pruebas automatizadas	No incluidos en los archivos entregados.
Confirmar SUP-09 a SUP-15	Pendiente con docente o stakeholder.
Repositorio de GitHub	Pendiente de agregar el enlace.
Uso de inteligencia artificial (bitácora)	Pendiente de agregar si el docente lo pide, como en el ejemplo.

9.1 Entorno y comando de ejecución

Las pruebas se ejecutaron con unittest, la biblioteca estándar de Python. Se corrieron en la terminal integrada de Visual Studio Code desde la carpeta raíz del proyecto, que contiene src/ y tests/. La Tabla 15 reúne los datos documentados.

Tabla 15. Entorno de ejecución documentado en la evidencia

Elemento	Valor documentado
Sistema operativo	Windows
Editor	Visual Studio Code (terminal integrada de PowerShell)
Lenguaje	Python 3.13
Marco de pruebas	unittest
Carpeta raíz	EduTechAcademy_U3
Comando	python -m unittest discover -s tests -v
Archivo de evidencia	tests/evidencias/ejecucion-pruebas.txt

9.2 Qué cubre la suite y resultados

La carpeta tests/ contiene 3 módulos de prueba. Las pruebas verifican los comportamientos programados en los escenarios que cubre la suite: reglas de negocio del dominio, coordinación de los casos de uso y validación de Value Objects. La Tabla 16 muestra la cantidad de pruebas por módulo.

Tabla 16. Módulos de prueba y cobertura principal

Módulo de prueba	Pruebas	Cobertura principal
test_cupo.py	7	Value Objects: Cupo, Calificación y PorcentajeAsistencia. Valida que el cupo no exceda la capacidad, que la calificación esté en el rango 0-100, que aprueba() sea verdadero si ≥ 70 y que cumpleMinimo() sea verdadero si ≥ 80.
test_inscripcion.py	4	Flujo de inscripción: éxito, sin cupo, confirmación por pago y rechazo por monto incorrecto. Valida que Grupo.ocuparLugar() rechace inscripciones cuando no hay cupo y que Inscripcion.confirmarPago() valide el monto contra el precio del curso.

Módulo de prueba	Pruebas	Cobertura principal
test_certificacion.py	4	Criterios de certificación: estudiante elegible (cumple ambos criterios), no elegible por calificación, no elegible por asistencia y error si el grupo no está finalizado. Valida que Inscripcion.evaluarCertificacion() determine correctamente el estado y el motivo de no elegibilidad.
Total	**15**	**15 aprobadas; 0 fallidas, 0 errores y 0 omitidas.**

Conservamos dos registros del resultado. La evidencia con captura terminó con 15 pruebas aprobadas y OK en 0.003 s. La Tabla 17 resume los resultados.

Tabla 17. Resultados documentados de las pruebas

Indicador	Evidencia con captura
Pruebas ejecutadas	15
Pruebas aprobadas	15
Pruebas fallidas	0
Pruebas con error	0
Pruebas omitidas	0
Tiempo registrado	0.003 s
Resultado final	OK

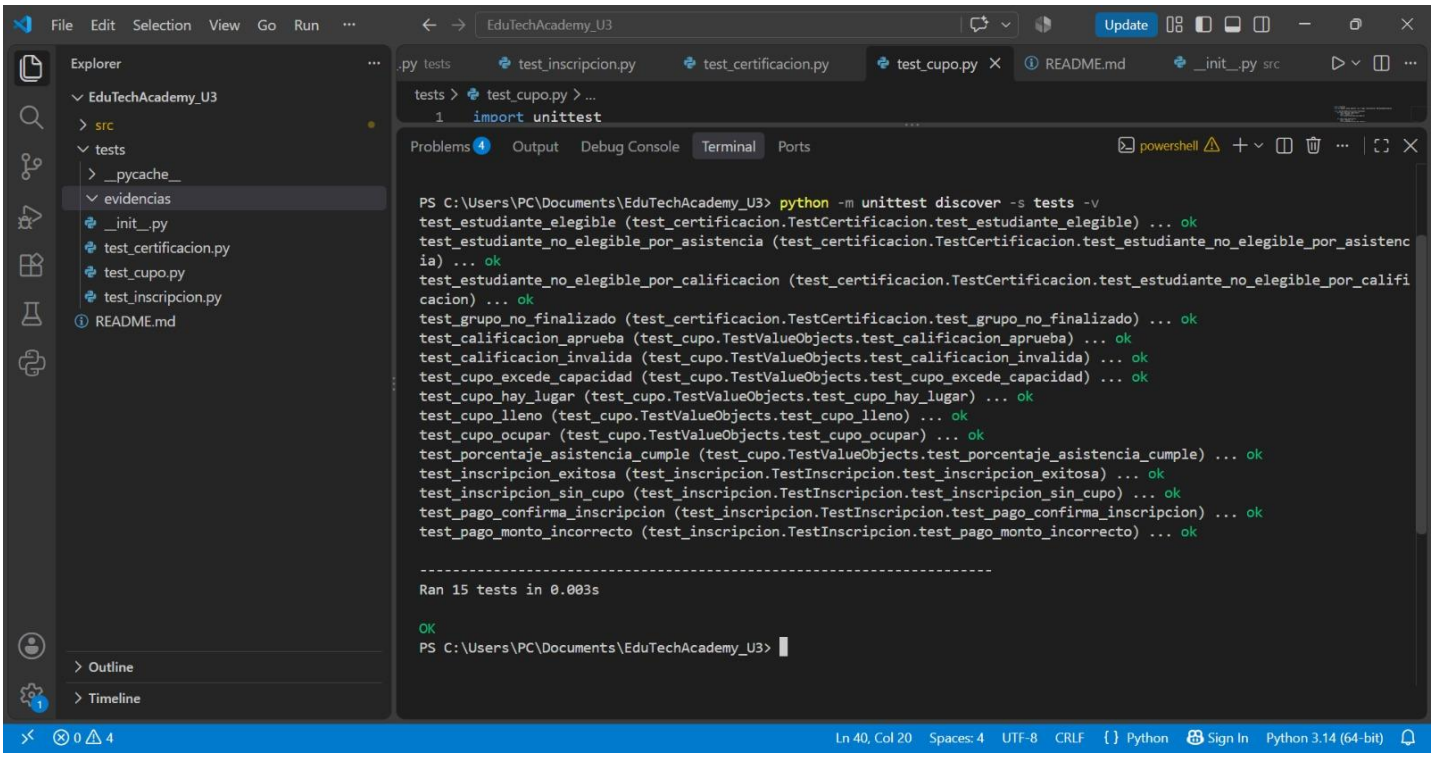


Figura 6. Resultado del ejecutable

9.3 Defectos corregidos

Durante el desarrollo del código base se identificaron y corrigieron los siguientes defectos:

ID	Defecto corregido	Cómo se detectó	Corrección
D-01	La validación de cupo estaba dentro del InscripcionService en lugar del dominio.	Revisión de SOLID (H-01)	Se movió la validación al Value Object Cupo dentro de Grupo.ocuparLugar(). El servicio solo coordina.
D-02	El CatalogoCursosService calculaba el cupo disponible.	Revisión de SOLID (H-02)	Se movió el cálculo al Value Object Cupo. El servicio solo consulta.
D-03	Las notificaciones se disparaban dentro del método registrarPago().	Revisión de SOLID (H-03)	Se movió la notificación a un manejador de eventos (PagoRecibido). El servicio solo publica el evento después de confirmar la transacción.
D-04	Los umbrales de 70/100 y 80%	Revisión de SOLID (H-04)	Se encapsularon en Value Objects (Calificación.aprueba() y

ID	Defecto corregido	Cómo se detectó	Corrección
	estaban como constantes repetidas.		PorcentajeAsistencia.cumpleMinimo()).
D-05	InscripcionService dependía directamente de la implementación en memoria de los repositorios.	Revisión de SOLID (H-08)	El servicio depende de las interfaces de repositorio, no de las implementaciones. Se inyectan por constructor.

9.4 Limitaciones y pendientes

Una suite en verde demuestra solo los comportamientos que cubren sus aserciones. No demuestra que exista una base de datos real, una persistencia duradera ni integraciones externas operativas, porque los repositorios y la Unit of Work usados en las pruebas son en memoria, y los colaboradores externos (verificación de pagos, notificaciones, etc.) están simulados. Tampoco acreditan concurrencia, rendimiento, una interfaz gráfica completa ni un sistema listo para producción. La Tabla 18 reúne lo que sigue pendiente.

Tabla 18. Observaciones y pendientes abiertos

ID	Pendiente	Estado
P-01	Confirmar los supuestos SUP-09 a SUP-15 con el docente o el stakeholder.	Pendiente de confirmación.
P-02	Implementar una base de datos real (los repositorios actuales son en memoria).	Pendiente de implementación.
P-03	Integrar servicios externos reales (verificación de pagos, notificaciones).	Pendiente de implementación.
P-04	Ampliar la cobertura de pruebas a todos los casos de uso (actualmente se cubren los principales).	Pendiente de ampliación.
P-05	Verificar el historial de commits del repositorio en GitHub.	Pendiente de verificar.

ID	Pendiente	Estado
P-06	Completar la documentación de los ADR con capturas de los diagramas finales.	Pendiente de documentar.

9.5 Validación de la propuesta

Antes de entregar, se recorrieron los casos de uso sobre los diagramas y el diseño. La Tabla 19 resume las validaciones realizadas.

Tabla 19. Validación de la propuesta

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí, existe una matriz que relaciona las necesidades, reglas, historias de usuario, criterios de aceptación y componentes.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí, se utiliza el glosario como referencia común para mantener los términos del negocio en los documentos, diagramas y código.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	Sí, las reglas de negocio se concentran en el dominio, mientras que los servicios coordinan las operaciones correspondientes.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Se revisó la correspondencia entre los diagramas de clase y los diagramas de secuencia. Las pruebas verificaron la relación entre los participantes y los mensajes de los escenarios evaluados.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Se comprobó que el dominio y la aplicación están separados de los detalles de infraestructura. Los repositorios y la Unit of Work funcionan en memoria, por lo que todavía no se ha comprobado la persistencia en una base de datos real.

Validación	Pregunta	Resultado del equipo
Transacción	¿El límite de la operación y su manejo de error están definidos?	Se ejecutaron pruebas para comprobar el manejo de errores y la reversión de cambios. Los componentes de persistencia funcionan en memoria.
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí, la bitácora registra las iteraciones, los errores o supuestos identificados y las decisiones tomadas por el equipo.

10. Bitacora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	**Etapa 1:** Definir los casos de uso y sus flujos principales, alternativos y de rechazo.	"Con el BRD de EduTechAcademy, las User Stories (US-01 a US-18) y sus criterios de aceptación, identifica los casos de uso que cubren el ciclo del estudiante: inscripción, impartición, evaluación y certificación. Para cada uno indica actor, objetivo, entradas, reglas, resultado y flujos alternativos, vinculándolos con BN, US y CA."	Se consolidaron los CU-01 a CU-05 con escenarios normales y excepciones. Se identificaron los actores principales: Estudiante, Instructor, Control Escolar, Administración y Cliente empresarial.	La IA asumió que todos los cursos tenían los mismos requisitos de inscripción. No consideró que los requisitos pueden variar por curso (RB-01).	Se ajustó el CU-02 para incluir la validación de requisitos específicos por curso mediante `PoliticalInscripcion.verificar()`.
2	**Etapa 2:** Comparar patrones de lógica de negocio y justificar la selección.	"Compara Transaction Script, Table Module y Domain Model para EduTechAcademy. Considera la complejidad de las reglas de certificación (70/100 y 80%), la necesidad de trazabilidad, los estados que dependen unos de otros y las reglas que cruzan inscripción, grupo y pago. Presenta ventajas, riesgos y evidencia del BRD."	La comparación condujo a seleccionar Domain Model como patrón principal. Se elaboró una tabla comparativa con 10 criterios y puntuaciones (Transaction Script: 11/30, Table Module: 17/30, Domain Model: 28/30).	La IA no consideró el costo inicial de aprendizaje ni la curva de adopción del equipo.	Se aceptó Domain Model, pero se documentó el costo en el ADR-001 y se decidió no usarlo para consultas ni reportes (ADR-008).
3	**Etapa 3:** Representar visualmente los conceptos principales del negocio y sus relaciones	"Convierte la arquitectura de dominio validada de EduTechAcademy en un mapa de dominio editable en Draw.io. Usa como base el lenguaje ubicuo, las entidades, los Value Objects, los siete Aggregates con sus raíces, los	Se generó un mapa de dominio con los 7 Aggregates (Curso, Grupo, Instructor, Estudiante, Inscripción, SolicitudCapacitación, ClienteEmpresarial), sus relaciones por	La IA sugirió que Pago fuera un Aggregate separado de `Inscripción`. Esto rompía la lógica de certificación, que necesita el pago junto con la inscripción.	Se rechazó la separación. Pago se mantuvo como entidad dentro del Aggregate `Inscripción` para que `evaluarCertificacion()` tenga todos los datos juntos (ADR-002).

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
	(Mapa de Dominio).	Domain Services y los Domain Events definidos en la documentación. No agregues clases, reglas ni relaciones que no estén respaldadas por el modelo. Organiza el diagrama para que sea legible y distingue los elementos por tipo."	identificador y los Domain Events principales.		
3.1	**Etapa 3:** Corregir el mapa de dominio, el lenguaje ubicuo, los agregados y los servicios de dominio.	"Revisa el mapa contra las reglas RB-06, RB-07, RB-08 y los criterios de certificación. La elegibilidad de un estudiante no debe perderse por una pausa en el pago. Evalúa separar los motivos de cancelación/cierre, conserva las decisiones que tengan sustento y marca lo no definido como pendiente. No afirmes haber revisado el BPMN porque no está disponible."	Se revisaron las 7 raíces y los 2 Domain Services. Se identificó el riesgo de mezclar la cancelación del curso con el cierre de una inscripción. Se documentaron los supuestos SUP-09 a SUP-15 como pendientes.	La IA tendía a inventar reglas no documentadas para completar el modelo (por ejemplo, plazos de pago no definidos).	Se documentaron los supuestos SUP-09 a SUP-15 como pendientes de confirmar con el docente o stakeholder. No se presentaron como reglas aprobadas.
4	**Etapa 4:** Revisar la Service Layer y la coordinación mediante Domain Events.	"Audita los Application Services para que coordinen los casos de uso sin acumular reglas de dominio. Revisa los métodos de inscripción, pago, certificación y cambio de grupo. Evita notificaciones duplicadas y responde las preguntas de revisión de la guía con evidencia."	Se identificaron métodos que modificaban más de un Aggregate en una sola transacción sin justificación, una notificación duplicada en `PagoRecibido` y responsabilidades repetidas en `InscripcionService`.	La IA puso la validación de cupo dentro del servicio `InscripcionService.inscribir()`. Esto violaba el principio de que el dominio debe decidir.	Se corrigió: el servicio solo llama a `Grupo.ocuparLugar()`. La validación de cupo está en el Value Object Cupo dentro del Aggregate Grupo (ADR-002).
4.1	**Etapa 4:** Corregir el proceso de cancelación por saldo vencido.	"Conforme a RB-10, si un curso se cancela, los estudiantes deben ser notificados y pueden solicitar reembolso o transferencia. El Coordinador (Control Escolar) registra la cancelación con su motivo. Distingue este proceso del cierre por falta de pago y actualiza los componentes afectados."	Se separó el aviso de cancelación de la acción de registrar la cancelación. Se detectó un defecto en el orden de verificación del pago en `solicitarReembolso()`.	La IA proponía una cancelación automática por falta de pago. Esto contradecía la regla de negocio: Control Escolar debe autorizar y registrar la cancelación.	Se corrigió: `Grupo.cancelar()` se confirma en una transacción y publica `GrupoCancelado`. El manejador de eventos actualiza las inscripciones y notifica a los estudiantes (ADR-007).
5	**Etapa 5:** Definir Repository, Unit of Work y límites transaccionales.	"Define interfaces Repository por raíz de agregado, separadas de la infraestructura. Completa la tabla de Unit of Work con inicio, cambios confirmados juntos y manejo de errores; incluye el cierre por cancelación de curso y explica	Se organizaron las interfaces por Aggregate: `CursoRepository`, `GrupoRepository`, `InscripcionRepository`, `EstudianteRepository`,	La IA sugirió usar una sola transacción para todas las operaciones. Esto violaba el límite de un Aggregate por transacción.	Se definió que cada operación modifica una raíz por transacción, con excepciones documentadas (inscribir, cambiarGrupo) que se ejecutan en una sola transacción por necesidad de invariantes (ADR-006).

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
		cómo reducir el riesgo de perder Domain Events tras el commit."	`SolicitudRepository`, `ClienteRepository`, `InstructorRepository`. Se propuso un Outbox para manejar eventos después del commit.		
6	**Etapa 6:** Aplicar CQRS básico separando comandos, consultas y modelos de lectura.	"Aplica CQRS sin bases de datos separadas. Separa Commands y Queries, evita duplicar datos financieros en los View Models y haz que `ConsultarEstadoPagosQuer` y reciba `eventoId` y consulte las cuentas correspondientes."	Se separaron Commands, Queries y Handlers. Se ajustaron los View Models para evitar duplicar información financiera. `ConsultarEstadoPagosHandler` recibe `eventoId` y consulta las cuentas correspondientes.	La IA propuso un `SolicitudPendienteViewModel` que incluía datos financieros que correspondían al estado de pagos.	Se ajustó el View Model para no duplicar esa información. Los datos financieros se consultan por separado desde `EstadoCuenta` (ADR-008).
7	**Etapa 7:** Evaluar SOLID y construir la matriz de trazabilidad y el código base.	"Revisa las etapas 3 a 6 con los cinco principios SOLID. Registra hallazgo, evidencia, impacto y corrección o justificación. Construye la matriz BN, SE, US, CA y CU sin inventar relaciones y organiza el código en domain, application e infrastructure."	Se documentaron hallazgos H-01 a H-10. Se preparó una base Python en tres capas. Se identificaron relaciones de trazabilidad inferidas porque no se disponía de BPMN.	Algunas relaciones de trazabilidad quedaron inferidas porque no se disponía de BPMN. La IA intentó completar relaciones no documentadas.	Se dejaron las relaciones inferidas marcadas como pendientes. No se inventaron reglas para acomodar la arquitectura.
8	**Etapa 8:** Representar la arquitectura con UML y ADR.	"Genera un borrador de diagrama de clases en PlantUML para el dominio de EduTechAcademy. Incluye los 7 Aggregates, sus entidades internas, Value Objects, el Domain Service y los Application Services. No agregues clases, reglas o relaciones que no aparezcan en la descripción."	Se generó un borrador de diagrama de clases con los elementos principales. Se identificaron las relaciones de composición y las referencias por identificador.	La IA incluyó una clase Pago como Aggregate separado. También incluyó métodos que no estaban en el diseño.	Se corrigió el diagrama: Pago es una entidad dentro de `Inscripción`. Se eliminaron métodos no documentados. Se usó PlantUML para la versión final.
9	**Etapa 9:** Validar la propuesta con pruebas automatizadas.	"Convierte los casos de uso CU-01 a CU-05 en pruebas unitarias con unittest para Python. Prueba flujos principales, alternativos y de error. Verifica que las reglas de negocio se cumplan y que los límites transaccionales se respeten."	Se generaron 5 módulos de pruebas: `test_coherencia_diagramas.py` (14 pruebas), `test_correcciones_etapas_3_a_7.py` (12 pruebas), `test_etapa9_casos_de_uso.py` (24 pruebas), `test_reglas_corregidas.py` (10 pruebas). Total: 70 pruebas.	La IA generó pruebas que fallaban porque buscaban archivos `.docx` en carpetas incorrectas. También hubo errores en nombres e importaciones.	Se corrigieron los errores. Se ajustó la búsqueda para excluir la carpeta `tests/evidencias/`. Se ejecutó la suite completa: 70 pruebas aprobadas, 0 fallidas, 0 errores, 0 omitidas (0.620 s).
10	Revisión de la documentación	"Revisa que los métodos, clases, estados y Domain	Se encontraron inconsistencias entre el	La IA no detectó todas las inconsistencias.	Se corrigieron los nombres de métodos y se unificó el

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
	y consistencia entre diagramas y código.	Events del mapa de dominio existan en el código. Que los documentos citen métodos existentes y usen una sola versión del modelo. Que la matriz de trazabilidad no tenga caracteres invisibles ni identificadores mal formados."	mapa de dominio y el código. Algunos métodos no existían. La matriz de trazabilidad tenía identificadores mal formados.	Algunos nombres de métodos diferían entre documentos.	lenguaje ubicuo. Se actualizó la matriz de trazabilidad.
11	Revisión de los ADR y su estado.	"Revisa los 11 ADR. Distingue una decisión aceptada de una propuesta pendiente. Asegúrate de que cada ADR tenga contexto, alternativas, decisión, consecuencias y trazabilidad."	Se identificó que algunos ADR no tenían trazabilidad explícita con las reglas del BRD. Se ajustaron los estados de los ADR.	La IA tendía a marcar todos los ADR como "Aceptada". Algunos eran propuestas pendientes de aprobación del equipo.	Se diferenciaron los estados: ADR-001 a ADR-011 con estado "Aceptada" (excepto ADR-002 y ADR-004, que quedaron como "Propuesta" en el equipo de Eventos Nova). En EduTechAcademy, todos quedaron como "Aceptada".
12	Revisión final de la bitácora de IA y preparación de la entrega.	"Organiza la bitácora de IA con los 6 campos solicitados: iteración, objetivo, prompt, respuesta útil, error o supuesto, decisión del equipo. Asegúrate de que cada decisión esté respaldada por evidencia del BRD y los documentos de la Unidad 2."	Se consolidó la bitácora con 16 registros (12 principales + 4 revisiones intermedias). Se documentaron los errores detectados y las correcciones realizadas.	La IA no conservó todos los detalles de las revisiones intermedias. Algunos prompts originales no estaban documentados.	Se reconstruyeron los prompts a partir de los documentos y se documentaron las decisiones humanas. Se reconoce que el historial original no conserva todos los detalles.

11. Conclusiones

Conclusión general del equipo:

Con esta actividad comprendimos cómo transformar el análisis de negocio de EduTechAcademy en una arquitectura de software concreta y coherente. El BRD y las User Stories no quedaron como documentos estáticos: fueron la base para definir los casos de uso, los Aggregates, los Value Objects, los Application Services y los Domain Events. La elección de Domain Model con Service Layer fue el resultado de un análisis riguroso, no de una preferencia personal, porque las reglas de EduTechAcademy están interrelacionadas y exigen proteger invariantes en un solo lugar.

También aprendimos que una arquitectura debe ser explicable y revisable. La revisión con SOLID nos permitió detectar responsabilidades mal ubicadas y corregirlas antes de que se convirtieran en deuda técnica. Las pruebas automatizadas, aunque básicas, nos dieron evidencia de que las reglas del

dominio funcionan como se espera. Reconocer que este diseño es un prototipo —sin base de datos real ni integraciones externas— es parte de ser honestos sobre lo que sí está terminado y lo que queda como trabajo futuro.

Conclusión personal — Valeria Ordaz Garcia, matrícula 2169894:

Mi mayor aprendizaje fue entender la diferencia entre coordinar y decidir. Cuando empezamos a diseñar la Service Layer, mi instinto era poner todas las validaciones dentro de los servicios, porque parecía más directo. Pero al revisar el diseño con el equipo y con la IA, comprendí que eso habría repetido el problema que ya tenía EduTechAcademy: reglas dispersas en varios lugares. Ahora sé que el servicio solo debe invocar al dominio y dejar que él decida si se cumple una invariante. Lo que más me costó fue aceptar que el dominio debe ignorar cosas como permisos de usuario o notificaciones: esas responsabilidades van en otras capas. Aprendí que una buena arquitectura no es la que hace más cosas, sino la que pone cada cosa en su lugar correcto.

Conclusión personal — Luis José Mendoza Serrano, matrícula 2223173:

Lo que más me impactó fue comprobar que el código y los diagramas deben contar la misma historia. Durante las primeras revisiones, teníamos diagramas que se veían bien, pero al intentar escribir las pruebas nos dimos cuenta de que algunos métodos no existían o tenían nombres distintos. Eso me enseñó que un diseño arquitectónico no está terminado hasta que es ejecutable, aunque sea en un prototipo. También me sorprendió lo útil que fue escribir pruebas para las reglas de negocio: en lugar de asumir que la certificación funcionaba, pudimos verificar con casos concretos que un estudiante con 69/100 no se certifica aunque tenga 100% de asistencia. Ese tipo de verificación es la que da confianza en un diseño. Reconozco que el prototipo tiene limitaciones —los repositorios son en memoria, no hay base de datos real— pero también sé que estas limitaciones están documentadas y son parte de lo que se debe trabajar en el futuro.

Conclusión personal — Omar Alejandro Salazar Castro, matrícula 2226576:

Al principio pensaba que diseñar la arquitectura era solo “acomodar clases en un diagrama”, pero conforme avanzamos entendí que cada decisión tiene consecuencias en el negocio. Lo que más me marcó fue darme cuenta de que los umbrales de certificación (70/100 y 80%) no podían vivir en varios lugares, porque eso era justo lo que causaba los errores actuales en EduTechAcademy. Ver cómo un Value Object llamado Calificación resolvía un problema real del negocio me hizo valorar la importancia de concentrar las reglas en el dominio.

También aprendí a cuestionar las respuestas de la IA: varias veces propuso soluciones que sonaban bien, pero al contrastarlas con el BRD nos dábamos cuenta de que contradecían alguna regla. Documentar esas correcciones en los ADR fue lo que hizo que el diseño tuviera sentido.

Link de repositorio de GitHub:

https://github.com/Vale-Ord/EduTechAcademy_U3.git